

Develop GenAI Apps with Gemini and Streamlit: Challenge Lab

AI Chef — Recipe Recommendation App
Built with Gemini, Streamlit, and Cloud Run

Prepared by: Luxna Ramnauth

Cymbal Health — Proof of Concept Report
September 2026

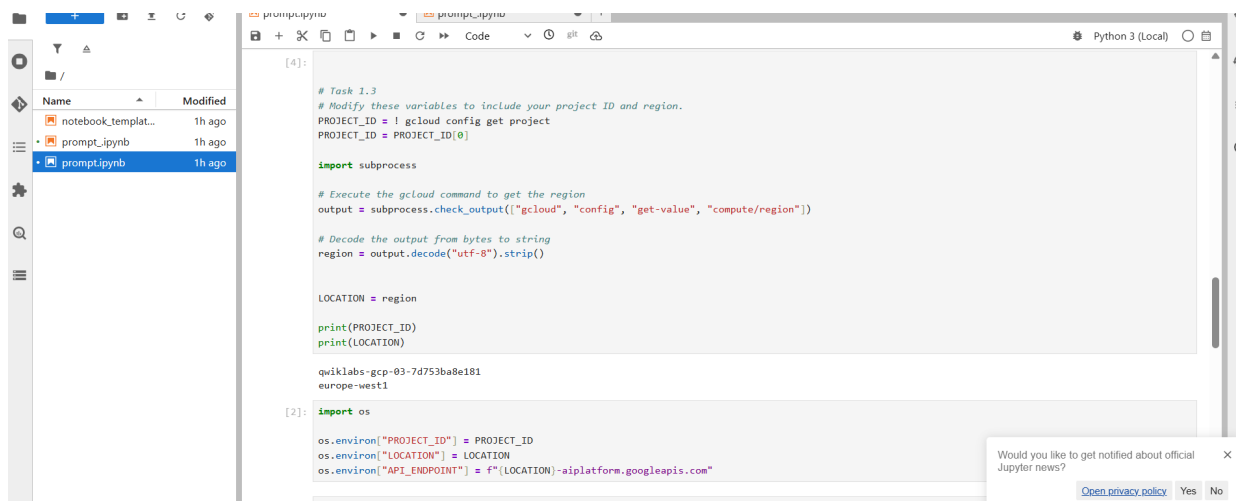
Project Overview

Cymbal Health is a regional health network focused on connecting affordable, high-quality care with everyday wellness. As part of a new initiative to improve customer nutrition, the team set out to build a proof-of-concept application called **AI Chef** — a tool that generates personalized recipe recommendations based on a customer's cuisine preference, dietary restrictions, food allergies, wine preference, and the ingredients they already have at home.

This challenge lab walked through the full lifecycle of building and shipping that proof of concept: testing connectivity with the Gemini API from a notebook, writing the Streamlit front end and the Gemini prompt logic in Python, running and debugging the app locally in Cloud Shell, containerizing it with Docker and pushing the image to Artifact Registry, and finally deploying the finished service to Cloud Run so it could be reached over a public HTTPS endpoint. Along the way, the project required diagnosing and fixing several real-world issues — malformed multi-line strings, an incorrect Gemini client argument, and an unreplaced model ID placeholder — which mirrors the kind of troubleshooting expected when moving a generative AI prototype from a notebook into a deployed, production-shaped service.

Task 1: Testing the Gemini API with cURL

Before writing any application code, the Gemini API endpoint was validated directly from a Workbench Jupyter notebook (prompt.ipynb). The notebook first resolved the active project ID and region using the `gcloud config` command-line tool, then exported them as environment variables so later cells (including a raw `curl` call to the `streamGenerateContent` endpoint) could authenticate against the correct Vertex AI project and location.



```
[4]: # Task 1.3
# Modify these variables to include your project ID and region.
PROJECT_ID = ! gcloud config get project
PROJECT_ID = PROJECT_ID[0]

import subprocess

# Execute the gcloud command to get the region
output = subprocess.check_output(["gcloud", "config", "get-value", "compute/region"])

# Decode the output from bytes to string
region = output.decode("utf-8").strip()

LOCATION = region

print(PROJECT_ID)
print(LOCATION)

quiklabs-gcp-03-7d753ba8e181
europe-west1

[2]: import os

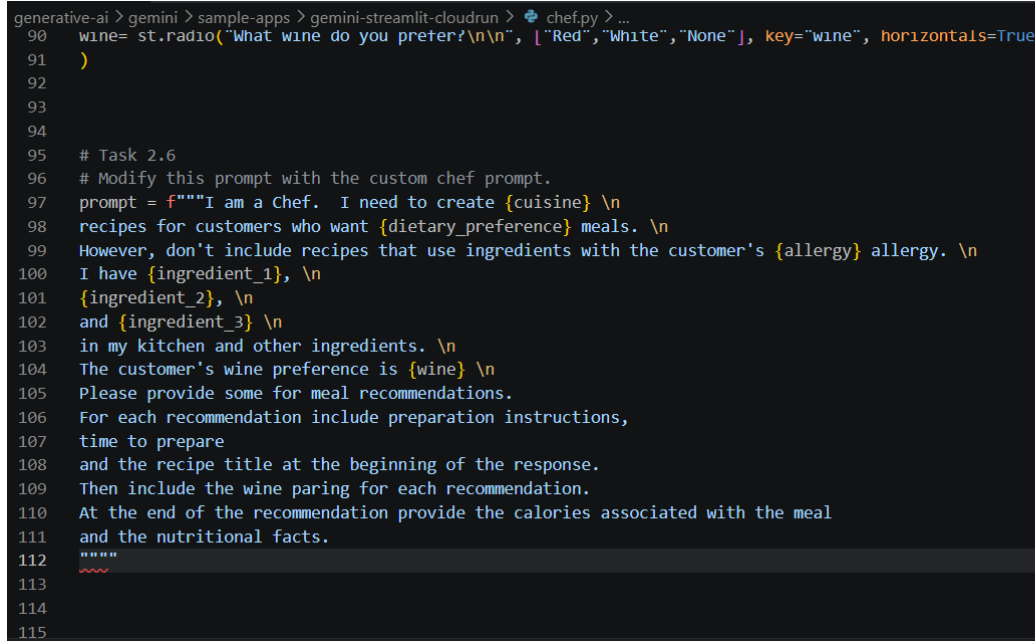
os.environ["PROJECT_ID"] = PROJECT_ID
os.environ["LOCATION"] = LOCATION
os.environ["API_ENDPOINT"] = f"{LOCATION}-aiplatform.googleapis.com"
```

Figure 1. `prompt.ipynb` resolving `PROJECT_ID` and `LOCATION` via the `gcloud` CLI, then exporting them as environment variables for the Gemini API call.

Code explanation: the notebook uses Python's subprocess module to shell out to `gcloud config get project` and `gcloud config get-value compute/region`, decodes the byte output to plain text, and stores the results in `PROJECT_ID` and `LOCATION`. These are then written into environment variables (`PROJECT_ID`, `LOCATION`, `API_ENDPOINT`) that the subsequent cURL cell reads when it builds the request URL to Vertex AI's `streamGenerateContent` endpoint. This confirmed early that credentials, project, and region were all correctly wired up before any application code was written.

Task 2: Streamlit UI and the Gemini Prompt (chef.py)

The `chef.py` file was cloned from the GoogleCloudPlatform `generative-ai` sample repository and extended in two places: a new Streamlit radio-button widget for wine preference, and a multi-line, template-driven Gemini prompt that stitches together every field the customer fills in.



```
generative-ai > gemini > sample-apps > gemini-streamlit-cloudrun > chef.py > ...
90 wine= st.radio("What wine do you prefer?\n\n", ["Red", "White", "None"], key="wine", horizontal=True
91 )
92
93
94
95 # Task 2.6
96 # Modify this prompt with the custom chef prompt.
97 prompt = f"""I am a Chef. I need to create {cuisine} \n
98 recipes for customers who want {dietary_preference} meals. \n
99 However, don't include recipes that use ingredients with the customer's {allergy} allergy. \n
100 I have {ingredient_1}, \n
101 {ingredient_2}, \n
102 and {ingredient_3} \n
103 in my kitchen and other ingredients. \n
104 The customer's wine preference is {wine} \n
105 Please provide some for meal recommendations.
106 For each recommendation include preparation instructions,
107 time to prepare
108 and the recipe title at the beginning of the response.
109 Then include the wine pairing for each recommendation.
110 At the end of the recommendation provide the calories associated with the meal
111 and the nutritional facts.
112 """
113
114
115
```

Figure 2. The wine-preference radio widget (`st.radio`) and the templated Gemini chef prompt in `chef.py`.

Code explanation:

`wine = st.radio("What wine do you prefer?", ["Red", "White", "None"], key="wine", horizontal=True)` — renders three mutually-exclusive radio options inline (thanks to `horizontal=True`) and stores the customer's selection in the `wine` variable, which is later interpolated directly into the Gemini prompt.

The `prompt` variable is an f-string that combines every collected input — cuisine, dietary preference, allergy, three ingredients, and wine — into a single natural-language instruction for Gemini. It explicitly asks the model to return, for each recipe: a title, preparation instructions and time, a wine pairing, and calorie/nutrition information at the end — giving the model a consistent output structure to follow.

```

89
90 wine= st.radio("What wine do you prefer?\n\n", ["Red","White","None"], key="wine", horizontal=True
91 )
92
93
94
95 # Task 2.6
96 # Modify this prompt with the custom chef prompt.
97 prompt = f"""I am a Chef. I need to create {cuisine} \n
98 recipes for customers who want {dietary_preference} meals. \n
99 However, don't include recipes that use ingredients with the customer's {allergy} allergy. \n
100 I have {ingredient_1}, \n
101 {ingredient_2}, \n
102 and {ingredient_3} \n
103 in my kitchen and other ingredients. \n
104 The customer's wine preference is {wine} \n
105 Please provide some for meal recommendations.
106 For each recommendation include preparation instructions,
107 time to prepare
108 and the recipe title at the beginning of the response.
109 Then include the wine paring for each recommendation.
110 At the end of the recommendation provide the calories associated with the meal
111 and the nutritional facts.
112 """
113

```

Figure 3. Full prompt block plus the GenerateContentConfig safety settings that follow it, controlling how strictly Gemini filters harassment, hate speech, and explicit content categories.

Debugging note: this block initially failed to run with a `SyntaxError: unterminated string literal` on the closing triple-quote. The cause was a hidden non-ASCII character (likely a smart/curly quote introduced by a copy-paste) inside the pasted text, which is invisible on screen but breaks Python's parser. The fix was to retype the block manually rather than pasting it, which avoided the hidden character entirely.

Task 3: Running and Debugging the App in Cloud Shell

With the code in place, the Python virtual environment was created, dependencies installed from `requirements.txt`, and the app launched locally with the Streamlit CLI, pointed at port 8080 to match what Cloud Run will later expect:

```
python3 -m venv gemini-streamlit
source gemini-streamlit/bin/activate
pip install -r requirements.txt

streamlit run chef.py \
  --browser.serverAddress=localhost \
  --server.enableCORS=false \
  --server.enableXsrfProtection=false \
  --server.port 8080
```

Running the app surfaced two further runtime errors before it worked end-to-end:

1. `TypeError: Client.__init__() got an unexpected keyword argument 'enterprise'` — the Gemini client was being constructed with `genai.Client(enterprise=True, project=PROJECT_ID, location=LOCATION)`. The Google Gen AI SDK has no `enterprise` parameter; the correct keyword to route calls through Vertex AI is `vertexai=True`.



Figure 4. `TypeError` raised by the incorrect `'enterprise'` keyword argument.

2. `google.genai.errors.ClientError: 400 INVALID_ARGUMENT — Invalid Endpoint name` — after fixing the client constructor, the app still failed because the model ID field in the code still held the literal placeholder text `GEMINI_FLASH_MODEL_ID` instead of an actual deployed model name (as instructed in Task 2: “Replace `GCP_PROJECT_ID` with Project ID, and `GEMINI_FLASH_MODEL_ID` with `model_id`”). This was resolved by replacing the placeholder with the real model identifier (e.g. `gemini-2.0-flash-001`).

```
CLOUD SHELL
Terminal (qwiklabs-gcp-03-7d753ba8e181) x +
[Open Editor] [Icons]

INFO:google_genai.models:AFC is enabled with max remote calls: 10.
INFO:google_genai.models:AFC remote call 1 is done.
INFO:httpx:HTTP Request: POST https://aiplatform.googleapis.com/v1beta1/projects/qwiklabs-gcp-03-7d753ba8e181/locations/global/publishers/google/models/GEMINI_FLASH_M
OBSERVATION:streamGenerateContentAlt-ess "HTTP/1.1 400 Bad Request"
2026-09-18 07:00:04.501 Uncaught app execution
Traceback (most recent call last):
  File "/home/student_03_215d38507f5b/generative-ai/gemini/sample-apps/gemini-streamlit-cloudrun/gemini-streamlit/lib/python3.12/site-packages/streamlit/runtime/scrip
trunner/exec_code.py", line 121, in exec_func_with_error_handling
    result = func()
    ~~~~~~
  File "/home/student_03_215d38507f5b/generative-ai/gemini/sample-apps/gemini-streamlit-cloudrun/gemini-streamlit/lib/python3.12/site-packages/streamlit/runtime/scrip
trunner/script_runner.py", line 640, in code_to_exec
    exec(code, module, dict)
  File "/home/student_03_215d38507f5b/generative-ai/gemini/sample-apps/gemini-streamlit-cloudrun/chef.py", line 147, in <module>
    response = get_gemini_flash_text_response(
               ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/student_03_215d38507f5b/generative-ai/gemini/sample-apps/gemini-streamlit-cloudrun/chef.py", line 38, in get_gemini_flash_text_response
    for response in responses:
  File "/home/student_03_215d38507f5b/generative-ai/gemini/sample-apps/gemini-streamlit-cloudrun/gemini-streamlit/lib/python3.12/site-packages/google/genai/models.py"
, line 5079, in generate_content_stream
    for chunk in responses:
  File "/home/student_03_215d38507f5b/generative-ai/gemini/sample-apps/gemini-streamlit-cloudrun/gemini-streamlit/lib/python3.12/site-packages/google/genai/models.py"
, line 3988, in _generate_content_stream
    for response_dict in self._api_client.request_streamed(
  File "/home/student_03_215d38507f5b/generative-ai/gemini/sample-apps/gemini-streamlit-cloudrun/gemini-streamlit/lib/python3.12/site-packages/google/genai/_api_clien
t.py", line 674, in request_streamed
    session_response = self._request(http_request, stream=True)
                      ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/student_03_215d38507f5b/generative-ai/gemini/sample-apps/gemini-streamlit-cloudrun/gemini-streamlit/lib/python3.12/site-packages/google/genai/_api_clien
t.py", line 572, in _request
    errors.APIError.raise_for_response(response)
  File "/home/student_03_215d38507f5b/generative-ai/gemini/sample-apps/gemini-streamlit-cloudrun/gemini-streamlit/lib/python3.12/site-packages/google/genai/errors.py"
, line 101, in raise_for_response
    raise ClientError(status_code, response_json, response)
google.genai.errors.ClientError: 400 INVALID_ARGUMENT. ('error': {'code': 400, 'message': 'Invalid Endpoint name: projects/qwiklabs-gcp-03-7d753ba8e181/locations/glob
al/publishers/google/models/GEMINI_FLASH_MODEL_ID.', 'status': 'INVALID_ARGUMENT'})
```

Figure 5. Traceback confirming the invalid endpoint caused by the unreplaced GEMINI_FLASH_MODEL_ID placeholder.

Once both fixes were applied, the app rendered correctly in the browser preview and returned recipe recommendations without errors.

AI Chef

What cuisine do you desire?

French



Do you have any dietary preferences?

Select your desired dietary preference.



Enter your food allergy:

peanuts

Enter your first ingredient:

ahi tuna

Enter your second ingredient:

chicken breast

Enter your third ingredient:

tofu

What wine do you prefer?

☒ Red ☐ White ☐ None

Generate my recipes.

Figure 6. The working AI Chef Streamlit interface, showing cuisine, dietary preference, allergy, ingredient, and wine-preference inputs, ready to generate recipe recommendations.

Tasks 4 & 5: Containerizing and Deploying to Cloud Run

With the application verified locally, the Dockerfile was updated to launch `chef.py`, and the image was built and pushed to a new Artifact Registry Docker repository:

```
export PROJECT=$(gcloud config get-value project)
export REGION=<lab region>
export AR_REPO=chef-repo
export SERVICE_NAME=chef-streamlit-app

gcloud artifacts repositories create $AR_REPO \
  --repository-format=Docker \
  --location=$REGION

gcloud builds submit \
  --tag "$REGION-docker.pkg.dev/$PROJECT/$AR_REPO/$SERVICE_NAME"
```

Finally, the containerized image was deployed to Cloud Run as a fully managed, publicly accessible service, with the project and region passed through as environment variables so the deployed container could reconstruct the same Vertex AI client configuration used during local testing:

```
gcloud run deploy $SERVICE_NAME \
  --port=8080 \
  --image="$REGION-docker.pkg.dev/$PROJECT/$AR_REPO/$SERVICE_NAME" \
  --allow-unauthenticated \
  --region=$REGION \
  --platform=managed \
  --project=$PROJECT \
  --set-env-vars=PROJECT=$PROJECT,REGION=$REGION
```

The `--allow-unauthenticated` flag exposes the service on a public HTTPS URL for demoing the proof of concept, while `--set-env-vars` ensures the running container has the same `PROJECT` and `REGION` values that `chef.py` reads at startup to initialize the Gemini client.

Summary

This challenge lab took the AI Chef proof of concept from an isolated API test in a notebook all the way to a deployed, publicly reachable Cloud Run service. The most valuable part of the exercise was not writing new code from scratch, but the troubleshooting workflow it demanded: reading Python tracebacks carefully to isolate the exact failing line, recognizing that a "generic" `SyntaxError` pointed to an invisible pasted character rather than a logic mistake, catching an SDK parameter name that had changed (`enterprise` vs. `vertexai`), and spotting a placeholder value (`GEMINI_FLASH_MODEL_ID`) that had been left un-substituted. Each of these mirrors the kind of debugging a developer regularly does when adapting sample code and shipping a generative AI feature into a real, deployed application for a business like Cymbal Health.